COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	October 19, 2026	**DATE SUBMITTED:**	October 20, 2026
NAME:	Uchi, Mao Sophie S. Verano, James Daniel M.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 13.1 **Parallel Algorithms and Multithreading**

1. PROCEDURE OUTPUT

ILO A & ILO B:

Part 1: Simple One-Threaded Example

Screenshot

Code

main.cpp > ...

```cpp
#include <iostream>
#include <thread>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();

    return 0;
}
```

Output

```
Printing integer: 10
Printing string: T.I.P.
```

Analysis	Following this Simple One-Threaded example, we can explore that the main function is executing a thread or "t1" that contains the void "print" function, an integer and a string. From the void "print" function, character outputs on what should be displayed from the terminal, which are integer n and string str. In this case, "10" will be the n's integer and "T.I.P." will be the str's string. As the thread executes, "t1.join()" is on hold in the meantime; it won't progress until the current running thread terminates, returning afterwards. **This algorithm's implementation is Concurrent, as the two queues wait for the single thread that shares the same printing location to finish displaying both integer and string at the same time, a long process of execution takes place as the thread operates by itself**.

Part 2: Multi-Threaded Example

Screenshot

Code

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <thread>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {
        "T.I.P",
        "Competent",
        "Computer",
        "Engineers"
    };
    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads){
        th.join();
    }
    return 0;
}
```

	Output
	[Uchi]
	`2: Computer 1: Competent 3: Engineers 0: T.I.P.`
	[Verano]
	`1 : Competent0 : T.I.P2 : Computer3 : Engineers`

Analysis	Following this Multi-Threaded example, we can examine main function containing a vector array called 's' with 4 unique strings, another vector array named 'threads' with a For Loop structure that will display the vector strings' size in a numerical order; inside the thread's parameter includes a void 'print' function, 'i' , and vector string's strings. From the void 'print' function, character outputs on what should be displayed from the terminal, which are integer n (employing 'to_string' to convert strings into integers), char "colon" and string str. Going back to the 'threads' vector thread, a push_back is also applied to pass arguments to the functions print, i and s[i]. This loop will continue to execute threads until the string vector doesn't have any more strings to display. Another For loop is applied, as the thread executes, "th.join()" is on hold in the meantime; it won't progress until the current running thread terminates, returning afterwards. **This algorithm's implementation is Parallel, as the queues wait for the single thread that shares a similar printing location, however this printing location is capable of working for multiple threads. Although the code works concurrently as well, there are no ways to tell which function displays first. Verifying the console output, the sequence is not properly arranged, resulting in a condition that races different functions on which output displays the earliest.**

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

Questions:	**Answers:**
1. Write a definition of multithreading and its advantages/disadvantages.	Multithreading in a program constructs an even smaller independent program/s that executes different tasks using thread. From a thread, combining multiple threads, creating a larger thread that manages multiple processes defines multithreading. The advantages would be the sharing

	of data resources from different threads as it ends in fusing outputs. Choosing Parallelism as its execution method, if the goal is only to finish computations in an instant. Choosing Concurrency as its execution method results in a better production rate, displaying a flourished output with no overtaking errors. The disadvantages would be choosing Parallelism as its execution method; this concept is remarkable, yet the downsides are heavy memory consumption by applying too many threads and its execution method is unpredictable due to threads racing each other to output first.
2. Rationalize the use of multithreading by providing at least 3 use-cases.	Combining the basic concept of the advantage of multithreading, it is the process where a device can handle multiple task execution with ease using threads. Most of multithreading are used for: 1st use case: **Heavy or High- performance data processing.** Can easily decompose data into more distinct component parts to make patterns easier to analyze. When handling massive datasets, there's always the worst case scenario where it is sequential processing O(N) or (NlogN) time complexity. But, with multithreading, with its capability to use thread, it can reduce the execution time, as its strategy is almost identical to merge sort. With multiple loads, the processing of data will be easier. 2nd use case: **graphical applications.** There's also Thread in GUI apps and Video games. Multi-threading is the one capable of rendering visual frames and listening for user inputs, also known as onclicks, at a smooth rate. It is still data. Since graphics loads asset files, complex calculation, or saving a project, multi-threading offloads the heavy workload to a known **background worker thread**. Which implies that the application processes the data silently in the background without you knowing it's already done. Lastly, 3rd use case: **High-Concurrency Network or known as servers.** Well, as always, its data. Network servers contain thousands of requests and data transmissions simultaneously. With multi-threading, the server instantly assigns a

	dedicated thread to handle each connection or request. Which means, this enables the server to process thousands of inputs concurrently maximizing data processing and efficiency.
3. Differentiate between parallelism and concurrency.	Parallelism and Concurrency are both executing concepts in multithreading, handling numerous tasks in different ways. For Concurrency, its goal is to deal with processing multiple tasks at hand during overlapping time periods, require less memory than parallelism, work on single and multicore systems, better production rate and less prone to mismatch execution of output. For Parallism, its goal is to deal with executing multiple tasks simultaneously, requires more memory than concurrency, can't work on a single-core system, better performance rate and prone to have an unsynchronized output due to the shared data.

Reference/s:

GeeksforGeeks. (2025, October 24). *Benefits of multithreading in operating system*. GeeksforGeeks. https://www.geeksforgeeks.org/operating-systems/benefits-of-multithreading-in-operating-system/

Libretexts. (2021, February 22). *4.4: Benefits of multithreading*. Engineering LibreTexts. https://eng.libretexts.org/Courses/Delta_College/Operating_System%3A_The_Basics/04%3A_Threads/4.4%3A_Benefits_of_Multithreading

Lawrence Livermore National Laboratory. (n.d.). *Introduction to parallel computing tutorial*. https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial

GeeksforGeeks. (2026, May 4). *Difference between concurrency and parallelism*. https://www.geeksforgeeks.org/operating-systems/difference-between-concurrency-and-pa

rallelism

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

- Create a global variable of type integer.
- Create an add function that will take an integer parameter and add that value to the global variable.
- Use multi-threading techniques to create 3 threads; individually pass the add function to the threads.
- Display the value of global variable at different combinations of using the join() per thread. Such that:
  - Display
  - T1.join()
  - Display
  - T2.join()
  - Display
  - T3.join()
- Provide an analysis based on the outputs of the multi-threading exercise.

Code

```cpp
#include <iostream>
#include <thread>
#include <chrono>

int global = 0;

// create an add function that will take an integer parameter
void add(int value) {
    //processing time
    std::this_thread::sleep_for(std::chrono::milliseconds(200));
    // add the value to the global variable
    global += value;
}

int main() {
    // create 3 threads passing the function and values
    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

    // display the value of global varialbe at different
    //combinations of using the join() per thread.
    std::cout << "First Display: " << global << std::endl;
    T1.join();

    std::cout << "Display after 1st join(): " << global << std::endl;
    T2.join();

    std::cout << "Display after 2nd join(): " << global << std::endl;
    T3.join();

    std::cout << "Display after 3rd join(): " << global << std::endl;

    return 0;
}
```

Output

```
First Display: 0
Display after 1st join(): 60
Display after 2nd join(): 60
Display after 3rd join(): 60
```

Analysis

Uchi's Analysis:
From the program above, the integer 'global' shares the same data inventory coming from individual pushed threads; the values of the 3 threads will be relevant as the 'add' function processes each working thread, updating the sum of the values by incrementing these

	arguments into the global parameter. After printing "First Display: ", threads sleeping by 200 milliseconds will overlap displays, as their sleeping cycle ends at the same time. **Theoretically after every join(), the global value shouldn't sum to 60 already, but the uncoordinated code instructs the threads to operate simultaneously, giving us the sum of 60 each display. Experimentally, after the "1st join", the undefined software bug will print the sum of 60, making the other join() unnecessary as the threads have already finished operating**.
	Verano's Analysis: As for the supplementary part B, we were tasked to create a C++ Code program and show a solution that specifically satisfies the given requirements. Firstly, the first requirement was to create a global variable that takes a type of integer. Which in the code above, line code 5, has already been initialized as "**Int global = 0**". That's very basic. Next was to create an "**add**" function that will take an integer as a parameter, and add that specific value that is inside the parameter to the global variable. This was initialized on **line code 8 - 13**. We have an "**add**" function that takes an integer and stores it into an int variable called "**value**", when the function is passed or called with the parameter, it has a processing time for the thread and it adds the value to the global variable. Now that we have the function, we only have to use multi-threading to create 3 threads; and individually pass the add function to the threads. After that, display the value of the global variable at different combinations using the **join()** function. Inside the main function, we can see in **line code 17 - 19**, is the initialization of the threads, and inside that calls the function and the parameter for the function. After passing the function, we display each thread one by one but after each **join()** function. The outcome turns out to be all the same. Why? Because when the add function is being processed, the **threads are simultaneously executing in parallel and sleeping at the same time**. The parallel Starts at the Threads, which are **created one after another almost instantly**. While the main thread is still waiting, all threads are already **finishing their 200 ms sleep at the exact same time**. So by the time that it wakes up, **the add function processes everything into the global**. Meaning, since the 200ms is done, all threads are finished therefore, it all processes in one function which is why it **instantly jumps to 60**.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

Screenshot

```cpp
#include <iostream>
#include "printerQueue.h"
#include <thread>
#include <chrono>

// Thread function for Batch 1:
// Handles all original Batch 1 jobs
void runBatchOne(Printer* printer1) {

    printer1->queueDelay();
    printer1->addJob(101, "Striker", 5);  // <--- This runs in parallel with #104
    printer1->queueDelay();
    printer1->addJob(102, "James", 12);
    printer1->queueDelay();
    printer1->addJob(103, "Jobert", 3);
    printer1->queueDelay();

    printer1->processJobs();
    printer1->timeDelay();
}

// Thread function for Batch 2:
// Handles all original Batch 2 jobs
void runBatchTwo(Printer* printer2) {
    // Initial offset delay so that it won't process at the same time
    // which might break the printer if it process at the same time.
    std::this_thread::sleep_for(std::chrono::milliseconds(200));

    printer2->queueDelay();
    printer2->addJob(104, "James", 25);   // <--- This runs in parallel with #101
    printer2->queueDelay();
    printer2->addJob(105, "Ely", 7);
    printer2->queueDelay();

    printer2->processJobs();
    printer2->timeDelay();
}
```

```cpp
int main() {
    // Create two separate instances so that they don't fight over the same pointers
    Printer printerBatch1;
    Printer printerBatch2;

    std::cout << "\n==================================================\n";
    std::cout << "   >>> Full Parallel Batch Processing (No Mutex) <<<\n";
    std::cout << "==================================================\n\n";

    // Start both batches at the exact same time
    std::thread T1(runBatchOne, &printerBatch1);
    std::thread T2(runBatchTwo, &printerBatch2);

    // Wait for both concurrent streams to completely finish
    T1.join();
    T2.join();

    // Final check loop matching your original code structure
    printerBatch1.processJobs();
    printerBatch2.processJobs();

    std::cout << "\n[--- All original assignment jobs completed successfully. ---]\n\n";
    return 0;
}
```

For Data racing, You only have to turn this off:

```
// which might break the printer if it process at the same time.
std::this_thread::sleep_for(std::chrono::milliseconds(200));
```

Output

With 200ms delay offset

```
==================================================
 >>> [STATUS] Processing Multiple Print Request... <<<
==================================================

[PRINTING] Processing Job #104 for User: James [25 pages]...

==================================================
 >>> [STATUS] Processing Multiple Print Request... <<<
==================================================

[PRINTING] Processing Job #101 for User: Striker [5 pages]...
[PRINTING] Processing Job #105 for User: Ely [7 pages]...
[PRINTING] Processing Job #102 for User: James [12 pages]...

[--- All pending print jobs completed successfully. ---]

[PRINTING] Processing Job #103 for User: Jobert [3 pages]...

[--- All pending print jobs completed successfully. ---]

[STATUS] No pending print jobs.

[STATUS] No pending print jobs.


[--- All original assignment jobs completed successfully. ---]
```

Without 200 ms delay offset

```
[ADDED] Job #[ADDED] Job #101 by Striker104 by James (25 pages) added to the queue.
 (5 pages) added to the queue.
[ADDED] Job #105 by Ely (7 pages) added to the queue.
[ADDED] Job #102 by James (12 pages) added to the queue.
[ADDED] Job #103 by Jobert (3 pages) added to the queue.

==================================================
 >>> [STATUS] Processing Multiple Print Request... <<<
==================================================

[PRINTING] Processing Job #104 for User: James [25 pages]...

==================================================
 >>> [STATUS] Processing Multiple Print Request... <<<
==================================================

[PRINTING] Processing Job #105 for User: Ely [[PRINTING] Processing Job #101 for User: Striker [5 pages]...
7 pages]...
[PRINTING] Processing Job #102 for User: James [12 pages]...

[--- All pending print jobs completed successfully. ---]

[PRINTING] Processing Job #103 for User: Jobert [3 pages]...

[--- All pending print jobs completed successfully. ---]

[STATUS] No pending print jobs.

[STATUS] No pending print jobs.
```

Analysis	**Uchi's Analysis:** From the code above, applying the program for Shared Printer Simulation using Queues from our previous activity to reconstruct it as a multithread. The goal was to handle different pending jobs in each batch and execute them simultaneously; this is where multithread enters the code, but how? There are two concurrent threads which are T1 and T2 located at line 49-50 that will process printing jobs with the separated batches at the same time with efficiency. After waiting for the both concurrent threads to finish, the jobs are ready to be displayed.
	Verano's Analysis: In the code provided, I've decided to use my printer queue Hands-on activity for multi-threading. So, where did I apply the multi-threading? Since in the HOA, it is about processing the amount of jobs added, in order for both batches to be executed simultaneously, I applied multi-threading in adding jobs as a batch. Adding thread in the batch printer, processes a job in batch one, at the same time, takes another job at batch two. Therefore, processing both batches at the same time, and processing them with efficiency. In the thread, once the job has in all batches been added, as T1 and T2, it waits both concurrent streams to complete before processing the jobs to be printed. And that's how I applied multi-threading in the printer queue that I have chosen since it best fits in processing jobs simultaneously achieving maximum efficiency.

Uchi's Conclusion

In conclusion, for this activity, I've learned multithreading, concurrency and parallelism; their difference from each other, trade-offs on why these approaches are important and their functionality as an individual system and concepts. For starters, this module demonstrates the techniques by giving metaphoric examples to better understand the works of these methods, and it's great with visualizations to prove that these practices are not too different from each other. As these strategies operate by handling numerous tasks, they also function as an individual technique, it is better to know the tiniest difference between them as the advantages and disadvantages depend on your primary objective. From the procedure, the example for One-Thread and Multi-Thread, it is better to analyse every line from these codes, as the first thread program example is easy to comprehend, yet the second thread program example I've struggled at–to my knowledge, a disadvantage in learning multithreading. As I've analysed the second multithread example, the line "threads.push_back(std::thread(print, i, s[i]))" has taken be aback, wondering on where the operation starts, but to my surprise, it is simpler to comprehend if you applied the single thread multiple times one-by-one individually, but from time to time, unsure if I truly fathom what I'm trying to bring out of my thoughts.

Verano's Conclusion

In my learning, **multi-threading has provided a lot of advantages in terms of data processing**. It is **known as a powerful tool for data-heavy algorithms** integrated with **searching and sorting algorithms**. As they divide processing loads across multiple CPU cores to provide a synchronous execution. Multi-threading is very well **known for its improvised execution speed and efficiency through concurrent processing**. In this activity, it successfully demonstrated the core concepts of multi-threading to better improve optimization to

different algorithms, and dynamic structures by using parallel search algorithms. Somehow, some of the library too also helped in organizing and illustrating how thread sleeping behaviors synchronizes their execution windows to process a result simultaneously (**Thread)** and (**chrono**).

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google AI Overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explanation of concept.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Part C concept clarity**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.